

Exceptions are objects. In the `except` clause, we handle the `AbortTransaction` exception that was raised at any lower level. We assign the value of the exception to the variable `error`. When we print that variable, the user will see the associated error message. Since the exception was handled in the `except` clause, the program continues running, and the user is asked what they wish to do.

Calling the Same Method on a List of Objects

Unlike in our bank example, in cases where individual objects do not need to be uniquely identified, using a list of objects works extremely well. Let's say you're coding a game and you need to have some number of bad guys, spaceships, bullets, zombies, or whatever else. Each such object will typically have some data it remembers and some actions it can perform. As long as each object does not require a unique identifier, the standard way to handle this is to create many instances of the object from the class and put all the objects into a list:

```
objectList = [] # start off with an empty list
for i in range(nObjects):
    oNewObject = MyClass() # create a new instance
    objectList.append(oNewObject) # store the object in the list
```

In our game, we represent a world as a large grid, like a spreadsheet. We want monsters placed at random locations in the grid. [Listing 4-12](#) shows the start of a `Monster` class with its `__init__()` method and a `move()` method. When a `Monster` is instantiated, it is told the number of rows and columns in the grid and the maximum speed, and it chooses a random starting location and speed.

File: `MonsterExample.py`

```
import random

class Monster()
    def __init__(self, nRows, nCols, maxSpeed):
        self.nRows = nRows # save away
        self.nCols = nCols # save away
```